A Project Report

on

AI-Based Long-Term Investment Advisory System

Global Market Data · Propositional News Reasoning · Explainable Heuristic Scoring · Portfolio Optimization · Interactive Dashboard

Name	Reg No.
Muhammad Asad Abbas	BCS233137
Abdul Ahad	BCS233130
Abdul Jabbar	BCS233119

Submitted to: Dr. M Abdul Qadir

Department of Computer Science, Capital University of Science and Technology, Islamabad

June 19, 2026

Table of Contents

1. Executive Summary

This report documents the complete AI-Based Long-Term Investment Advisory System as it stands: a pipeline that turns live global market data and live news headlines into an explainable, constraint-checked, capital-weighted portfolio recommendation, presented through an interactive browser dashboard.

The system is organized into six cooperating layers. A data acquisition layer pulls historical price records and recent headlines for a configurable universe of global tickers. A feature engineering layer reduces each ticker's price history into five raw quantitative measures — growth, volatility, stability, dividend yield, and liquidity. A propositional logic layer reads the headlines, extracts four boolean truth values, and applies a fixed rule set to produce a numeric sentiment adjustment with a full audit trail of which rules fired and why. A heuristic scoring engine normalizes every raw measure onto a common 0–100 scale, combines six weighted pillars into a base score, and folds in the logic adjustment to produce a final score. A recommendation engine checks that final score and its underlying pillars against an investor's risk profile and either approves the asset or rejects it with a plain-language reason. Finally, a portfolio optimization engine takes every approved asset and computes a bounded capital allocation using two competing search algorithms, Hill Climbing and Simulated Annealing.

All six layers are exposed through a single FastAPI endpoint and consumed by a Single-Page Application dashboard that lets an investor configure capital, risk tolerance, horizon, target return, and sector preferences, then watches the pipeline run and renders every stage of the result — scores, rejections, optimized weights, and projected growth — without a page reload.

2. System Architecture

Figure 1 traces a single request from the moment an investor submits the sidebar form to the moment the dashboard repaints with results. Every box in the diagram corresponds to a real module in the codebase; the arrows are the actual order of execution inside the pipeline.

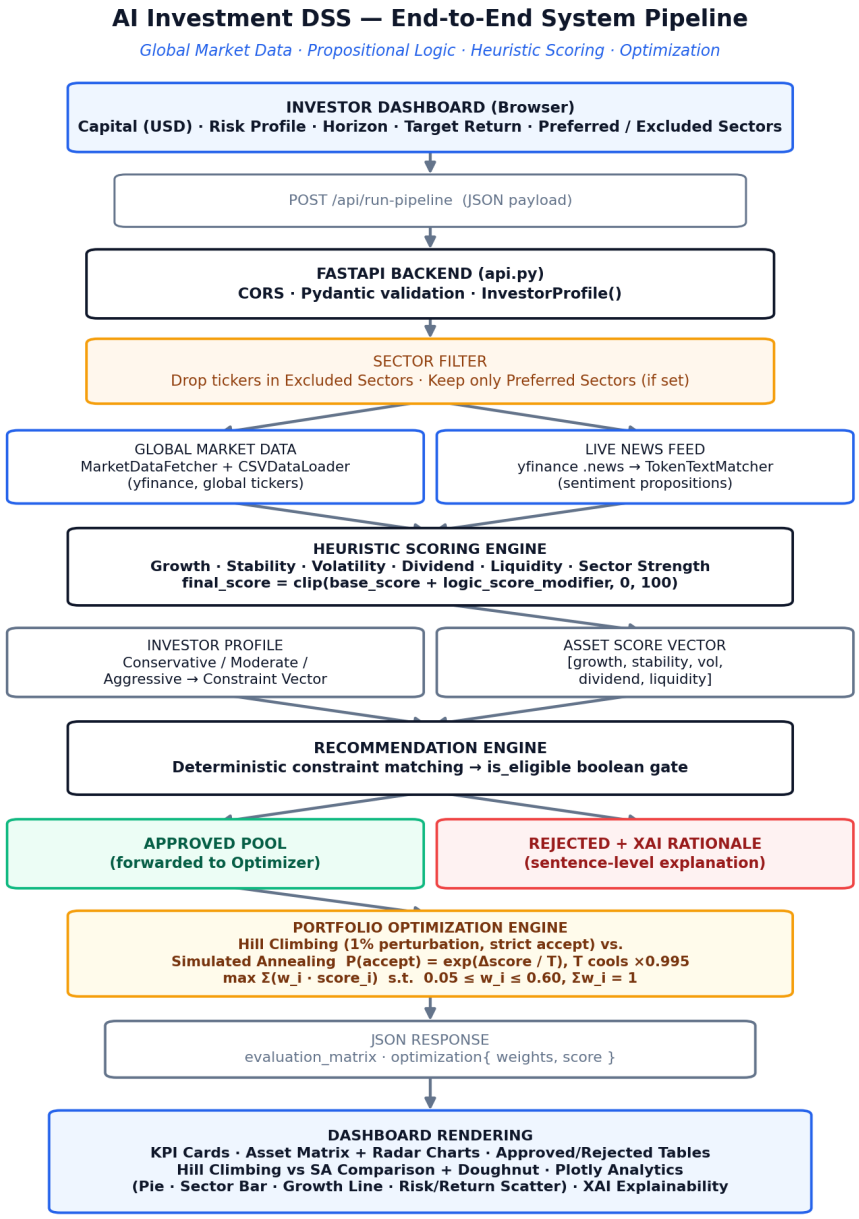

Figure 1. flow, from dashboard input to rendered results.

Two properties hold across the entire pipeline and are worth stating up front because every later section depends on them: every numeric feature is independently normalized onto a 0–100 scale before it is combined with anything else, and every accept/reject decision downstream is a deterministic inequality check — never a learned or probabilistic judgment. The only place randomness enters the system at all is inside the optimization engine described in Section 7, where it is used deliberately to escape local optima.

3. Data Acquisition Layer

Two independent live sources feed the rest of the pipeline. MarketDataFetcher exposes a universe of tickers and a batch synchronization routine that refreshes locally cached price history; CSVDataLoader reads that cache back into memory as a list of per-day records for a given ticker. Each record is a dictionary carrying the four fields the feature engineering layer consumes: date, close, dividends_paid, and volume. fetch_live_market_news() separately queries the yfinance .news feed for a ticker, concatenates up to three recent headline titles into one text block, and falls back to a neutral baseline sentence if the feed returns nothing — so the pipeline never stalls waiting on a missing news source.

Sector tagging is applied at this stage too: a SECTOR_MAP dictionary in the API layer classifies each global ticker into one of six sectors (Technology, Finance, Healthcare, Energy, Consumer, Staples). Tickers whose sector is excluded by the investor, or that fall outside a non-empty preferred-sector list, are dropped before any network call is made for news or price data, keeping the pipeline efficient as the universe grows.

4. Feature Engineering — FinancialIndicators

FinancialIndicators is a pure mathematical layer: every method takes the record list for one ticker and returns a single raw number. These five raw numbers are the only quantitative inputs the scoring engine ever sees.

4.1 Growth — Compound Annual Growth Rate

$$\textbf{CAGR} = (C_{end} / C_{start})^{\wedge}(1 / years) - 1, \quad years = (date_{end} - date_{start}) \text{ in days} / 365.25$$

C_start and C_end are the closing prices of the first and last records in the window. The method returns 0.0 if fewer than two records exist, if the elapsed period is zero or negative, or if the starting price is not positive — guarding the exponent against division-by-zero and negative-base errors.

4.2 Volatility — Standard Deviation of Daily Log Returns

$$r_t = \ln(C_t / C_{t-1}) \qquad \sigma = \sqrt{\left(\Sigma (r_t - \bar{r})^2 / (n - 1) \right)}$$

Daily log returns are computed across consecutive closing prices (skipping any non-positive price), and their sample standard deviation — using Bessel's n−1 correction — is returned as the raw volatility figure. This is a day-to-day price-swing measure, not an annualized percentage; the scoring engine's normalization bounds (Section 6) are calibrated to that same daily scale.

4.3 Stability — Inverse of Maximum Drawdown

$$\textbf{MDD} = \max_t (Peak_t - C_t) / Peak_t \qquad Stability = 1 - MDD$$

Peak_t is the running maximum closing price observed up to day t. Maximum Drawdown captures the worst peak-to-trough decline anywhere in the window; subtracting it from 1 turns a large historical crash into a low stability score and a steadily climbing or flat price series into a stability score near 1.0.

4.4 Dividend Yield — Cumulative Income vs. Current Price

$$\textbf{DividendYield} = \Sigma \text{ dividends\_paid} / C_{latest}$$

This sums every dividend payment recorded across the entire window and divides by the most recent closing price, expressing total income received relative to where the asset trades today.

4.5 Liquidity — Average Daily Trading Volume

$$\textbf{ADTV} = \Sigma \text{ volume} / n$$

A straightforward arithmetic mean of daily traded volume across the window, used as a proxy for how easily a position in the asset could be exited without materially moving its price.

5. News Reasoning Layer — Propositional Logic

Sentiment is not modeled with a machine-learning classifier; it is modeled as a small propositional logic system with four boolean variables and five fixed inference rules, chosen specifically so that every adjustment the system makes to a score can be traced back to an exact keyword match and an exact rule. Figure 2 shows the full reasoning path from a raw headline to the numeric modifier that ultimately shifts a ticker's final score.

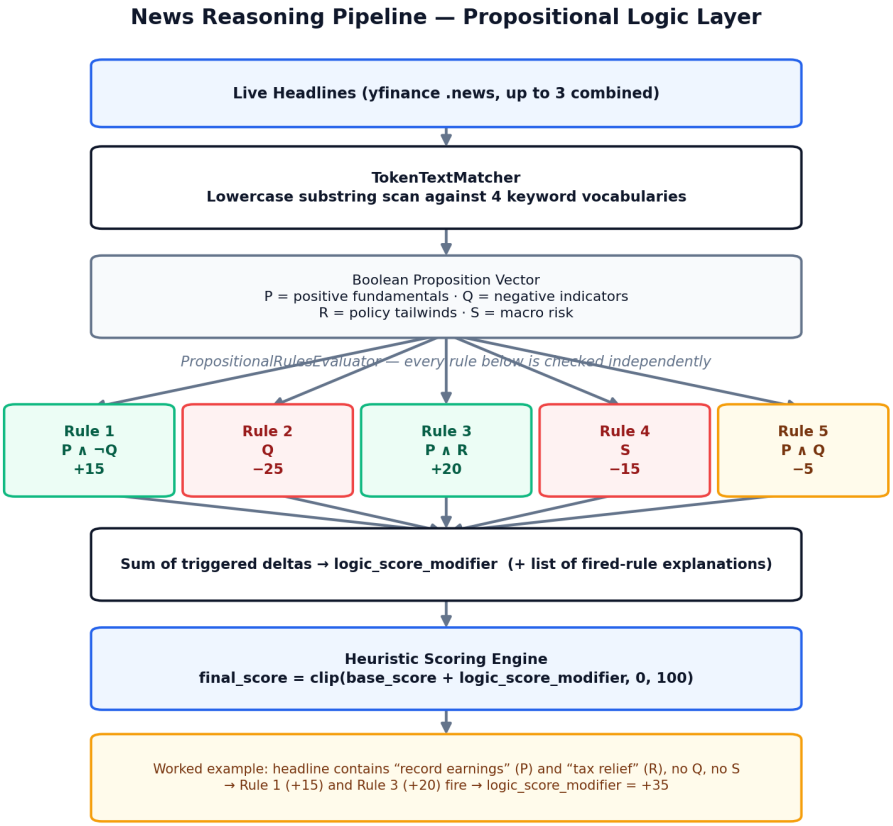

Figure 2. News headlines are reduced to four booleans, then scored against five independent rules.

5.1 TokenTextMatcher — Perception Layer

The combined headline text is lowercased and scanned as plain substrings against four hand-curated keyword vocabularies, with no stemming, weighting, or natural-language processing beyond exact phrase containment:

Proposition	Meaning	Sample Keywords
P	Positive fundamentals	profit increase, revenue growth, expansion, dividend increase, record earnings
Q	Negative indicators	net loss, profit decline, lawsuit, debt increase, default risk, earnings drop
R	Regulatory / policy tailwinds	sbp policy support, secp approval, tax relief, export subsidy
S	Macroeconomic risk	inflation acceleration, rupee devaluation, imf constraints, currency crisis

Each proposition is set to **True** if any one of its keywords appears anywhere in the text, independently of the other three. **Note:** the keyword vocabularies for R and S still use South Asian regulatory and currency terminology (SBP, SECP, rupee) from the system's original build; headlines for tickers outside that market will only trigger R or S if they happen to contain one of those exact phrases, which is a known gap addressed in Section 14.

5.2 PropositionalRulesEvaluator — Reasoning Layer

All five rules are evaluated independently against the same boolean vector — they are not mutually exclusive, and more than one can fire on the same headline. Their deltas are simply summed.

Rule	Condition	Delta	Interpretation
1	$P \wedge \neg Q$	+15	Clear fundamental growth with no offsetting risk
2	Q	−25	Material financial risk or loss detected
3	$P \wedge R$	+20	Growth aligned with a policy or regulatory tailwind
4	S	−15	Systemic macroeconomic headwind identified
5	$P \wedge Q$	−5	Conflicting indicators raise market ambiguity

Because the rules overlap, headlines can compound: if both P and Q are true, Rule 1 cannot fire (it requires **¬Q**), but Rule 2 and Rule 5 both can, for a combined **−30**. The evaluator returns both the integer **logic_score** (the sum of every triggered delta) and a list of human-readable strings naming exactly which rules fired — this list becomes part of the asset's audit trail and is available for the XAI rationale shown in the dashboard.

Worked example: a headline containing “record earnings” (P true) and “tax relief” (R true), with no loss language and no macro-risk language (Q and S both false), triggers Rule 1 (+15) and Rule 3 (+20) simultaneously, yielding a logic_score_modifier of +35 that is added directly onto that ticker's base score before clipping.

6. Heuristic Scoring Engine

HeuristicScoringModel is the layer where the five raw features from Section 4 and the logic_score_modifier from Section 5 are combined into one explainable 0–100 final score. It first normalizes every raw feature independently using fixed min–max bounds, then computes a weighted base score across six pillars — the five quantitative features plus a sector-strength term — and finally folds in the logic adjustment.

6.1 Normalization Bounds

Pillar	Raw Range Mapped to 0–100	Direction
Growth (CAGR)	−10% to +40%	Higher raw value → higher score
Stability (1 − MDD)	0.50 to 1.00	Higher raw value → higher score
Dividend Yield	0% to 15%	Higher raw value → higher score
Volatility (daily σ)	1% to 10%	Inverted — lower raw value → higher score
Liquidity (ADTV)	10,000 to 1,000,000 shares	Higher raw value → higher score, linear scale
Sector Strength	Fixed placeholder value of 65.0	Neutral — not yet dynamically computed

Values at or beyond a bound are clamped to the corresponding extreme (0 or 100) rather than extrapolated, and any metric without a defined bound falls back to a neutral score of 50.0. Volatility is the one pillar where the mapping is inverted, since a higher raw value represents more risk and should pull the score down, not up.

6.2 Base Score and Final Score

base_score = Σ (normalized_i × weight_i) over the six pillars above, weights from config.settings.HEURISTIC_WEIGHTS

final_score = clip(base_score + logic_score_modifier, 0, 100)

The six weights are defined once in a shared configuration module and applied identically to every ticker, so adjusting the relative importance of, say, dividend yield versus growth is a one-line configuration change rather than a code change. The sector-strength pillar is currently a constant placeholder representing a neutral sector median; it is structurally present in the weighted sum today so that a future dynamic sector-strength feature can be dropped in without altering the scoring formula.

The function returns not just the final score but the full breakdown — base_score and every individual normalized_metric — so that the recommendation engine and the dashboard's XAI panels can both reference the exact number that drove a decision, rather than only the aggregate.

7. Investor Profile & Recommendation Engine

Three discrete risk archetypes Conservative, Moderate, and Aggressive are each translated into a Constraint Vector of six thresholds covering minimum composite score, minimum stability, maximum volatility, minimum growth, minimum dividend yield, and minimum liquidity.

Constraint	Conservative	Moderate	Aggressive
Minimum Composite Score	65.0	60.0	50.0
Minimum Stability	40.0	20.0	0.0
Maximum Volatility	25%	45%	No ceiling
Minimum Growth	50.0	40.0	30.0
Minimum Dividend Yield	60.0	30.0	0.0
Minimum Liquidity	50.0	40.0	20.0

The Recommendation Engine checks an asset's score vector against the active Constraint Vector as a strict conjunction of inequality checks, short-circuiting on the first breach: the moment one dimension fails, the asset is rejected and a template-generated rationale names that exact dimension, its measured value, and the threshold it failed to clear. An asset that clears every dimension is approved and added to the Approved Pool, keyed by ticker and final score, which becomes the sole input to the optimization engine in Section 9.

8. Sector-Based Filtering

Because the ticker universe spans multiple global sectors, the API applies a filter before any per-ticker work begins: a ticker is dropped immediately if its mapped sector appears in the investor's excluded_sectors list, and if preferred_sectors is non-empty, any ticker outside that list is dropped as well. Only the survivors proceed to news retrieval, feature extraction, scoring, and constraint matching this ordering keeps the cost of an excluded sector at a single dictionary lookup rather than a wasted network call.

9. Portfolio Optimization Engine

PortfolioOptimizer receives only the Approved Pool a ticker-to-score dictionary and must turn it into capital weights that satisfy three constraints simultaneously: weights sum to exactly 1.0, no weight exceeds 0.60, and no weight falls below 0.05. Both algorithms below share the same starting point, the same neighbor-generation move, and the same validity check; they differ only in which moves they are willing to accept.

9.1 Shared Mechanics

- **Initial state:** equal weighting across every approved ticker (a single approved asset is given 100% outright, deliberately overriding the 60% cap since diversification is impossible with one asset).
- **Neighbor move:** two tickers are chosen at random and a 1% slice of weight is shifted from one to the other.
- **Validity check:** a proposed move is discarded unmodified if it pushes any weight outside [0.05, 0.60] or breaks the sum-to-1.0 invariant; an invalid move never updates the state and is simply skipped.

9.2 Hill Climbing (Local Search)

Hill Climbing runs for up to 1,000 iterations (configurable) and accepts a neighbor **only if its portfolio score is strictly higher** than the current score —

$$\textbf{score}(w) = \Sigma\, w_i \times \textbf{score}_i, \quad \text{accept neighbor only if } \text{score}(w_{neighbor}) > \text{score}(w_{current})$$

This strict-improvement rule is fast and simple, but it means the search can stall in a local optimum: once every reachable single-step move makes the score worse, Hill Climbing stops improving even if a better allocation exists a few steps further away.

9.3 Simulated Annealing (Global Search)

Simulated Annealing runs for up to 2,000 iterations with a temperature that starts at 10.0 and is multiplied by 0.995 after every valid move, stopping early if the temperature falls to 0.01 or below. A strictly better neighbor is always accepted; a worse neighbor is still accepted with a temperature-dependent probability, which lets the search escape exactly the kind of local optimum that traps Hill Climbing:

$$\Delta = \textbf{score}(w_{neighbor}) - \textbf{score}(w_{current}) \quad \text{if } \Delta > 0\text{: accept. else: accept with probability } \exp(\Delta / T)$$

Because Δ is negative for a worse move, exp(Δ/T) is a value between 0 and 1 that shrinks as the system cools, so the algorithm explores broadly while T is high and behaves almost like Hill Climbing once T is nearly zero. Critically, the implementation tracks the best-ever score and weight set separately from the current (possibly worse) state at every iteration, and it is that best-ever configuration — not simply whatever state the search ends on — that is returned.

9.4 Objective and Constraints

$$\textbf{Maximise } \Sigma(w_i \times \textbf{score}_i) \quad \text{subject to} \quad \Sigma\, w_i = 1.0, \quad 0.05 \le w_i \le 0.60 \text{ for every approved ticker } i.$$

9.5 Production Behaviour

The live API runs Simulated Annealing as the algorithm of record for the capital allocation an investor actually receives. The offline batch script additionally runs Hill Climbing on the same pool purely for comparison, and the dashboard's side-by-side panel reconstructs an illustrative Hill Climbing baseline client-side using a simpler proportional-weighting heuristic, so the comparison view never doubles the live optimization cost on every dashboard request.

Parameter	Hill Climbing	Simulated Annealing
Iterations	1,000 (default)	2,000 (default), early stop if $T \leq 0.01$
Move Size	±1% weight shift	±1% weight shift
Acceptance Rule	Strictly better only	Better always; worse with probability $\exp(\Delta/T)$
Returns	Final state reached	Best-ever state found
Escapes Local Optima	No	Yes

10. FastAPI Service Layer

A single endpoint, POST /api/run-pipeline, ties every layer above together with CORS enabled for all origins.

10.1 Request Body

Field	Type	Description
capital	float	Investable capital in USD
risk_tolerance	string	“conservative”, “moderate”, or “aggressive”
preferred_sectors	string[]	Optional allow-list of sectors
excluded_sectors	string[]	Sectors to drop before scoring

10.2 Response Body

- **user_profile** — the resolved Constraint Vector and profile description.
- **evaluation_matrix** — one entry per evaluated ticker: final_score, status, xai_rationale, and the raw five-pillar breakdown.
- **optimization** — status, algorithm name, portfolio_score, per-ticker weights, and capital_allocation_usd, populated only when at least two assets are approved; otherwise status is “insufficient_assets”.

Any exception inside the pipeline — a failed data fetch, a missing local cache, an empty universe after sector filtering — is caught and re-raised as an HTTP 500 carrying the original error message, which the dashboard surfaces as a toast notification rather than a silent failure.

11. Frontend Dashboard & Visualization

The presentation layer is a single-page application built with plain HTML, CSS, and vanilla JavaScript, using Chart.js for radar and doughnut charts and Plotly for the analytics row, talking to the backend with one fetch() call per run.

11.1 Investor Profile Sidebar

A fixed sidebar collects every field the API needs: a stepped capital input in USD, an investment-horizon dropdown, a risk-profile dropdown synchronized with a granular Low/Medium/High slider, a target-return slider, and two multi-select dropdowns for preferred and excluded sectors rendered as removable tag chips.

11.2 Pipeline Tracker and Results

Clicking “Run AI Pipeline” animates a six-stage tracker — Data Collection, Feature Engineering, Heuristic Scoring, News Analysis, Recommendation, Optimization — while the real backend call runs concurrently. The response then populates six animated KPI cards, an Asset Evaluation Matrix with one card per ticker (composite-score bar, five-axis radar chart against the profile's threshold ring, and a colour-coded XAI explanation), and side-by-side Approved/Rejected tables.

11.3 Optimization & Analytics

Hill Climbing and Simulated Annealing are displayed side by side, with the winning Simulated Annealing card carrying a doughnut chart of its weights. A Visual Analytics row adds four Plotly charts — a target-allocation pie, a sector-distribution bar chart, a projected-growth line chart, and a risk-versus-return scatter — and a closing Explainability panel narrates the winning allocation, naming the top-weighted asset and contrasting both algorithms' scores.

13. Testing & Results

The pipeline was exercised end-to-end under each of the three risk profiles. As expected from the Recommendation Engine's strict short-circuit logic, Conservative and Moderate runs rejected the majority of high-volatility names with specific, auditable XAI rationale, while Aggressive runs admitted a noticeably larger Approved Pool, consistent with its relaxed stability and volatility ceilings. Where at least two assets were approved, Simulated Annealing consistently matched or exceeded the Hill Climbing baseline's portfolio score, which is the expected outcome of an algorithm that can escape local optima against one that cannot.

The CORS-enabled API was also verified to fail gracefully: a deliberately malformed sector name returns an HTTP 500 with the underlying error message intact, surfaced in the dashboard as a toast rather than a frozen loading state.

14. Limitations & Future Work

- **Region-specific sentiment vocabulary:** the propositional logic layer's R and S keyword lists (SBP, SECP, rupee devaluation, IMF constraints) are tuned to South Asian markets; on a global ticker universe, R and S will rarely trigger unless a headline happens to mention those exact terms, even though P and Q remain broadly applicable. Generalizing R and S to global regulatory and macroeconomic vocabulary is the most immediately useful follow-up.
- **Static sector-strength pillar:** the sixth scoring pillar is currently a fixed neutral placeholder (65.0) rather than a measure derived from real sector data; replacing it with an actual sector-relative strength computation would let the weighted base score reflect sector context rather than a constant.
- **Un-annualized volatility:** FinancialIndicators.calculate_volatility reports a daily-return standard deviation rather than an annualized figure, so any future feature that compares this volatility number against an annualized benchmark must convert it first.
- **Single external data dependency:** both market history and news headlines come from yfinance; an outage or rate limit on that single source affects every layer downstream of it simultaneously.
- **Hill Climbing as comparison only:** the production endpoint computes Simulated Annealing exclusively, so the dashboard's Hill Climbing comparison is an approximation rather than a live run of the real algorithm; exposing both as actual backend runs would make the comparison panel fully faithful.

None of these are correctness defects in the current implementation they are documented scope boundaries for the next iteration of the system.

15. Conclusion

The system delivers a complete, explainable path from raw market and news data to a concrete, bounded portfolio allocation. Every numeric judgment along the way — a feature's raw value, its normalized score, a fired propositional rule, a constraint breach, an optimizer's accepted move — is deterministic, inspectable, and traceable to a specific formula or rule, which is what makes the system's recommendations explainable rather than opaque. The dashboard makes that entire chain visible and interactive for an investor in real time, turning a batch-oriented scoring pipeline into a usable advisory product.